Measure AI by Motion, Not by Head

GTM AI cost containment and ROI framework · Oct 12, 2026 · @Andy P

Executive summary

Fund motions, not heads: measure AI by what the spend produces, not by who spends it — and cost control and ROI become the same number.

The current plan is role-based dashboards and per-person spend caps. The intent is right: enable people on Claude Code without runaway cost. But a per-person cap rations innovation. The people doing the most valuable AI work hit the limit first, and a cap can't tell waste from breakthrough.

The proposal. Keep the same gateway data, but organize it around *motions* — real GTM workflows with a business object attached — instead of around people.

- **One join key.** Every piece of AI work carries a Jira ticket. The ticket maps to a Salesforce campaign, its opportunities, and accounts.
- **One measurement layer.** A derived ledger in Databricks joins gateway usage to those business objects and rolls cost up by motion.
- **One control model.** Control comes from visibility into value, not limits on individuals. Role-based views still exist — every ticket has an owner — so nothing the current plan wants is lost.

The proof. One real motion, rebuilt end to end: a trade-show list turned into a running campaign. It checks what we already own before paying to enrich, loads clean into Marketo, ties to a Salesforce campaign, and every token is traceable to that campaign.

The ask. Two weeks to build that one proof on a real list, plus read access to the gateway usage logs. Then decide together whether to scale it.

> **One line to remember:** AI is the lever, the motion is the unit, the campaign is where cost and return meet.

Why per-person caps backfire

A cap answers "who spent" — it cannot answer "what did it produce," so it throttles value and waste equally.

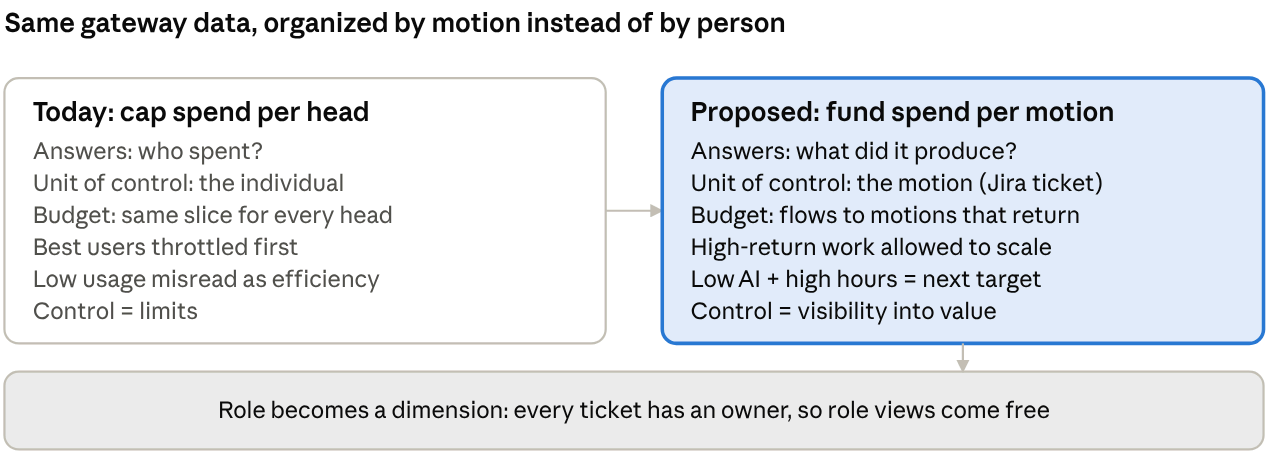
Failure mode	What happens under a per-person cap
Punishes the best users	The person doing the most exploratory, valuable work hits the limit first and is throttled when most productive.
Blind to value	A dollar on a pipeline-generating campaign and a dollar on idle noodling look identical.
Funds winners and losers equally	Every head gets the same safe slice, regardless of return.
Drives usage underground	People ration, hide usage, or route work to tools outside the gateway — making the data worse.
Misreads low usage as good	Someone using no AI looks like a model citizen, but may be doing the work by hand at far higher cost.

The underlying error: treating AI spend as a *cost to contain per head* instead of an *investment to allocate per outcome*. Caps are the instrument you reach for when you are blind. The fix is sight, not a smaller budget.

What we see today: spend per model, and nothing else. No per-user, per-tool, or per-purpose view, and no way to tie a single dollar to a campaign.

The framework: fund motions, not heads

Motion-first measurement is strictly more powerful than role-first: role views fall out of it for free, but the reverse is impossible.



Role-based caps vs. motion-based funding

The shift is in the organizing principle, not the data. The gateway already records the usage; we change what it is grouped by.

Four principles

1. **Measure by motion, not by tool or role.** A motion is a real workflow with a business object attached (a campaign, an opportunity).
2. **Check what we own before we pay to re-learn it.** Every enrichment checks the Databricks account and contact tables before buying from ZoomInfo.
3. **Tie every dollar to an outcome.** Every campaign carries an owner, a budget, start and end dates, and a goal.
4. **Control is visibility, not limits.** Keep a loose guardrail during transition; retire hard caps as motion-level visibility comes online.

What low AI usage really signals. It is a question, not a virtue. A motion with little AI and many human hours is either being done by hand — the expensive way — or happening in a tool outside the gateway. Either way, it tells us where to look next.

How measurement works

Measurement happens in a derived Databricks ledger that joins gateway usage to business objects through one key: the Jira ticket. Databricks is the only place that can see both halves — AI usage on one side, campaign and revenue data on the other.

```
flowchart LR
  P["Project folder<br/>CLAUDE.md + settings.json<br/>declares ticket GTM-123"] --> S["Claude Code session"]
  S -->|"model calls carry a tag header<br/>(jira_key = GTM-123)"| G["Databricks AI Gateway"]
  G --> U[("Gateway usage table<br/>tokens + request tags")]
  S -->|"runs"| R["Pipeline scripts"]
  R -->|"API calls"| Z["ZoomInfo"]
  R -->|"writes, tagged GTM-123"| L[("Motion run log<br/>credits used, internal matches,<br/>records loaded")]
  U --> M[("Motion ledger<br/>derived table")]
  L --> M
  J["Jira ticket<br/>campaign + opportunity links"] -.->|"join at query time"| M
  C["Salesforce campaign<br/>owner, budget, dates, goal"] -.->|"join at query time"| M
  M --> V["Genie space + dashboard"]
```

The four stops of an ID: declared once in the local project file → attached to every outbound call as request metadata → logged by the gateway → extracted into the ledger and joined to Jira and Salesforce.

The non-negotiable rule: the ID rides around the model, not through it

The model never reads or writes the authoritative ticket key. It travels as request metadata (a header), not as prompt text — so the model cannot mangle it and context truncation cannot drop it. The same principle applies to bulk data: records stay in Databricks and are processed with set-based SQL; the model only orchestrates.

Verified mechanics (Oct 12, 2026)

Piece	What the documentation confirms	Source
Per-request tags in Databricks	The Unity AI Gateway accepts a `Databricks-Ai-Gateway-Request-Tags` HTTP header holding a JSON map of string keys and values, and records it in the `system.ai_gateway.usage` system table.	Databricks usage tracking

Piece	What the documentation confirms	Source
Older gateway variant	AI Gateway on model serving endpoints records a `usage_context` map from the request body in `system.serving.endpoint_usage` (10 KiB limit).	Configure AI Gateway
Who can query usage	Both usage tables are admin-only by default — we need a grant or a filtered view.	Same pages
Tagging from Claude Code	`ANTHROPIC_CUSTOM_HEADERS` adds custom headers to every model request; it can be set in a project's `.claude/settings.json` `env` block and is reapplied mid-session when the file changes.	Claude Code environment variables

Put together: a per-project settings file can stamp every model call with the ticket key, with no prompt text involved. **Which gateway variant we run is the first thing to confirm**, because it decides whether the tag goes in a header or in the request body.

The ledger query, in sketch

```
-- Cost of AI per motion: tokens grouped by the ticket tag
SELECT
  request_tags['jira_key']           AS jira_key,
  SUM(input_tokens + output_tokens)  AS total_tokens,
  COUNT(*)                           AS requests
FROM system.ai_gateway.usage
WHERE request_tags['jira_key'] IS NOT NULL
GROUP BY 1;
-- Then join jira_key -> Jira ticket -> Salesforce campaign,
-- and add ZoomInfo credits and hours from the motion run log.
```

Column names in this sketch are illustrative; confirm them against the real table schema before building.

Validation: every key in the ledger is checked against a real Jira ticket and Salesforce record. Anything unmatched is flagged, not counted.

Known boundary: anything that routes around the gateway is invisible to this telemetry. The Atlassian MCP already runs outside it — which is fine, because Jira is a dimension we join on, not a source we meter. Mapping what else bypasses the gateway defines the true edge of what we can measure.

Data model

Every call stamps exactly one thing — the Jira key; everything else is reached by walking the chain at query time. One stamp, infinite cuts: by campaign, by owner, by team, by role, by motion type.

```
erDiagram
  JIRA_TICKET ||--o{ GATEWAY_USAGE : "tags"
  JIRA_TICKET ||--o{ MOTION_RUN_LOG : "tags"
  JIRA_TICKET }o--|| SF_CAMPAIGN : "links to"
  SF_CAMPAIGN ||--|| MARKETO_PROGRAM : "syncs 1:1"
  SF_CAMPAIGN ||--o{ SF_OPPORTUNITY : "influences"
  SF_OPPORTUNITY }o--|| SF_ACCOUNT : "belongs to"
  JIRA_TICKET {
    string jira_key PK
    string owner
    string motion_type
    string sf_campaign_id FK
    string sf_opportunity_id FK
  }
  GATEWAY_USAGE {
    timestamp request_time
    string requester
    map request_tags
    int input_tokens
    int output_tokens
  }
  MOTION_RUN_LOG {
    string run_id PK
    string jira_key FK
    int records_in
    int matched_internal
    int enriched_external
    int zoominfo_credits
    int records_loaded
    float human_hours
```

```
}
SF_CAMPAIGN {
  string campaign_id PK
  string owner
  float budget
  date start_date
  date end_date
  string goal
}
MARKETO_PROGRAM {
  string program_id PK
  string sf_campaign_id FK
}
SF_OPPORTUNITY {
  string opportunity_id PK
  string account_id FK
  float amount
  string stage
}
SF_ACCOUNT {
  string account_id PK
}
```

Design decisions behind the model

- **Ticket as the join key, not a Salesforce ID.** It decouples work from systems: a call never needs to know whether it serves a campaign, an opportunity, or a Marketo program. One enforceable rule — all AI work ties to a ticket — instead of five.
- **Pointer local, truth remote.** The local project file holds only the pointer (this run = ticket X). The ticket-to-campaign relationships stay in Jira and Salesforce and are resolved by joins, so nothing goes stale on someone's laptop.
- **Campaigns and opportunities carry the dollars.** Accounts are reached through opportunities; they are not stamped directly.
- **Tag shape stays flat:** object type, object ID, optional human-readable name. Event tagging, not a config tree.

Open question: when marketing and sales both touch an opportunity, does each get credit (influenced vs. sourced)? The ledger can support either, but the rule must be agreed before numbers are shown.

The proof motion: a trade-show list, rebuilt

One real list, run through a governed path, proves cost containment and ROI on the same campaign. It was chosen because everyone recognizes the pain and it touches every system: Databricks, ZoomInfo, Marketo, Salesforce.

Today: someone returns from a trade show with a ragged spreadsheet — inconsistent columns, personal emails or none. It goes through ZoomInfo blind, gets hand-massaged into Marketo, and attribution is an afterthought. Three months later, nobody can say what the trade show produced.

Governed:

```
flowchart TD
  A["1. Intake + normalize<br/>raw file to a standard staging table<br/>in Databricks"] --> B["2. Check internal first<br/>match on email, fall back to<br/>company + name"]
  B --> C{"3. Split into buckets"}
  C -->|"already owned"| E["5. Load to Marketo<br/>one clean push into the program"]
  C -->|"partial"| D["4. Enrich the gap only<br/>ZoomInfo API, batched"]
  C -->|"genuinely new"| D
  D -->|"write enriched records back"| W[("Databricks account<br/>and contact tables")]
  D --> E
  E --> F["6. Attribution wiring<br/>Marketo program to Salesforce campaign<br/>owner, budget, dates, goal"]
  W -.->|"next list matches more internally"| B
```

Step	Tool	What makes it governed
1. Intake + normalize	Databricks	One standard schema, regardless of who built the spreadsheet
2. Check internal first	Databricks SQL (written by Claude Code)	Deterministic, parameterized queries — Genie is for human exploration, not the production path
3. Split into buckets	Databricks	The split *is* the cost-containment number: "matched X% internally, paid for the rest"

Step	Tool	What makes it governed
4. Enrich the gap	ZoomInfo API	Only unmatched records are bought; results are written back so the warehouse compounds
5. Load	Marketo via MCP	No logic in Marketo — it is a dumb endpoint for a finished list
6. Attribution	Marketo ↔ Salesforce campaign sync	Owner, dollars, dates and goal set before the first touch, not after

Scope guard: these are proof-of-concept scripts on one machine, not a hardened production system. Present them as the pattern, not the product.

ROI model

The ROI of AI is not the token cost — it is the cost and time AI removes from the motion. Tokens alone are cheap and make a weak story, so we measure the whole motion.

Cost side — total motion cost

```
\text{Motion cost} = \text{AI tokens} \times \text{gateway rate} +
\text{ZoomInfo credits spent} \times \text{credit rate} + \text{human hours}
\times \text{loaded hourly rate}
```

Alongside it, report the cost *avoided*: ZoomInfo credits not spent because records matched internally.

Return side — three honest numbers

1. **Total motion cost** — from the ledger, as above.
2. **Goal attainment** — the campaign's own stated goal (defensible; it was set before launch).
3. **Influenced pipeline** — opportunities tied to the campaign, explicitly labeled an estimate. Saying so builds credibility.

The comparison that makes the case: same list, two ways. The proof fills this table with real figures; until then the cells stay empty rather than guessed.

Measure	Manual way (today)	Governed way (proof)
Records in		
Matched internally (no purchase)		
ZoomInfo credits spent		
Human hours, list to live campaign		
AI tokens (now visible per campaign)		
Days from show to first touch		
Tied to a Salesforce campaign with goal		
Influenced pipeline (estimate)		

Reading the ledger across motions

Pattern	What it means	Action
High AI, low hours, goal met	The model to replicate	Fund more of it
Low AI, high hours	Done by hand, or in a tool we can't see	Next target for AI
High AI, no business object	Untethered spend	Question it
High AI, goal missed	Spend without return	Fix or stop the motion

Rolling it out: the Claude Code toolkit

A single GTM plugin puts identical rules, motions, and controls on every desktop, so the governed path becomes the easiest path. People drop manual lists when one command is easier than building a spreadsheet — compliance becomes the lazy option.

```
flowchart TB
  PL["GTM plugin<br/>installed once per desktop"] --> CM["CLAUDE.md
tiers<br/>what Claude knows"]
  PL --> SK["Skills<br/>what Claude can do on request"]
  PL --> SA["Subagents<br/>who Claude delegates hard parts to"]
  PL --> SC["Slash commands<br/>how the person asks"]
  PL --> HK["Hooks<br/>what happens automatically"]
  PL --> MC["MCP config<br/>Marketo, Salesforce, Databricks via gateway"]
  PJ["Per-project settings<br/>ticket tag header"] --> TAG["Every model call
tagged"]
  SC --> SK
  SK --> MC
  HK -.->|"blocks untagged work"| SK
```

Mechanism	Job in this design	Pull or push
`CLAUDE.md` (global tier)	Standing rules: check internal first, every motion needs a ticket, IDs never go in prompts	Always loaded
`CLAUDE.md` (project tier)	Which motion this folder serves; points at the ticket	Always loaded
Project `.claude/settings.json`	Sets `ANTHROPIC_CUSTOM_HEADERS` so every model call carries the ticket tag	Automatic
Skills	Packaged motions, e.g. *enrich a trade-show list*; author once, everyone runs it the same way	Pull
Subagents	Delegated workers for stages that need their own reasoning, e.g. resolving messy match conflicts	Pull
Slash commands	Front doors, e.g. `/enrich-list GTM-123`	Pull
Hooks	Enforcement that cannot be forgotten	Push
MCP config	Wires the systems; ZoomInfo and Marketo calls go through code that also writes the run log	Automatic
Plugin	The container that installs all of the above identically	Install

Correction to the earlier hooks plan: tagging is a setting, not a hook

The documentation shows hooks act on *tool calls* (allow, deny, rewrite inputs, add context). They do not add headers to the *model requests* the gateway meters. The tag therefore lives in a settings `env` value (`ANTHROPIC_CUSTOM_HEADERS`), and hooks do what they are good at: refusing ungoverned work.

Hooks inventory (event names as documented)

#	Hook	Event	What it does	Priority
1	Ticket required	`PreToolUse`, matched to ZoomInfo and Marketo write tools	Denies the call when the project has no valid ticket; set `onFailure: "block"` so a broken hook fails closed	Core
2	Session context	`SessionStart`	Reads the project file, adds the ticket and campaign as context, and exports `JIRA_KEY` for scripts via `CLAUDE_ENV_FILE`; warns if missing (this event cannot block)	Core
3	Command gate	`UserPromptExpansion`, matched to `/enrich-list`	Blocks the motion command when no ticket is declared	Core
4	Check internal first	`PreToolUse`, matched to ZoomInfo	Denies enrichment when the run log shows no internal match step for this run	Next
5	Write-back	`PostToolUse`, matched to ZoomInfo	Appends credits used to the run log and confirms records were written back to Databricks	Next
6	Run summary	`SessionEnd`	Flushes the run log row for the motion	Later

Where hooks live: a plugin ships them in `hooks/hooks.json`; teams can also define them in `~/.claude/settings.json`, a project's `.claude/settings.json`, or organization-wide managed settings, which admins can lock with `allowManagedHooksOnly` (hooks reference).

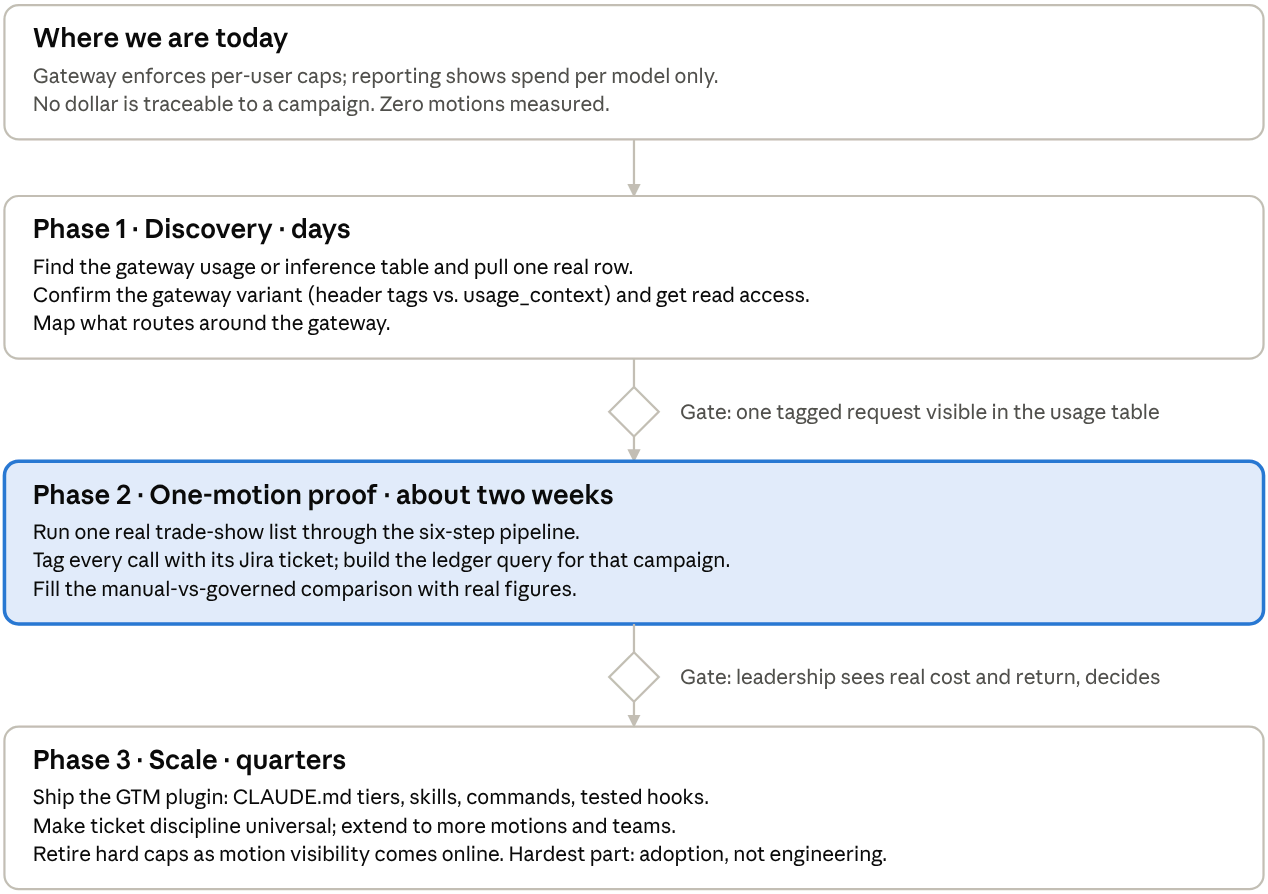
```
gtm-plugin/
  .claude-plugin/plugin.json
  hooks/hooks.json            # hooks 1-6
  scripts/                    # hook and pipeline scripts
  skills/enrich-list/SKILL.md
  agents/match-resolver.md
  commands/enrich-list.md
  templates/CLAUDE.md         # global rules tier
  templates/project/          # project CLAUDE.md + settings.json with the
tag header
```

Caution: hooks fire on every matching event, so a bad hook breaks everyone's workflow at once. They are the most powerful and most dangerous piece, and they ship only after the one motion is proven by hand.

Where we are vs. where we need to be

We are at the starting line technically, but the first proof point is about two weeks away. The full vision is a multi-quarter effort whose hardest part is adoption.

The proof is two weeks away; the full rollout is quarters



Roadmap · 3 phases, 2 gates

The shape of this gap is the argument for starting small: judge the idea on the short distance first, and commit to the long one only once the proof is real. Fund the two-week proof, not the transformation.

Explicitly parked for later: a governed org-wide account table, a local knowledge-wiki architecture, org-wide telemetry dashboards, and semantic caching. Caching belongs to the platform team, and only if the ledger shows a meaningful share of spend is near-duplicate prompts.

Delivery kit

Lead with the reframe, prove it with one campaign, gesture once at the horizon, and stop. Slides 2–5 carry the weight; restraint on slide 6 is what reads as leadership.

Elevator pitch (60 seconds)

> We're about to cap AI spend per person to control cost. But capping people doesn't control cost — it rations innovation, and our best users hit the limit first. I think we flip it: measure AI by what the spending produces, not who spends it. Take one real workflow — turning a trade-show list into a running campaign — and rebuild it so it checks what we already own before we pay to enrich, loads clean, and ties every token to a campaign through its Jira ticket. Do that, and cost control and ROI stop being two separate worries. They're the same number. I've built one example on a real campaign — happy to walk anyone through it.

Six-slide outline

#	Slide title	The one point it makes	Time
1	Who's spending vs. what it's producing	Caps ration innovation; role views fall out free from a motion model	2 min
2	One campaign, the way it works today	Ragged list, blind ZoomInfo spend, hand-loading, attribution lost	2 min
3	The same campaign, governed	Check what we own, enrich the gap, load clean, tie to a campaign	3 min
4	The cost AI removes, not the cost AI adds	Before/after: hours saved, enrichment avoided, tokens now visible	3 min
5	Where cost and return finally meet	Campaign with owner, budget, dates, goal; three honest numbers	2 min
6	One motion is the pattern	Ticket → campaign → opportunity → account; gesture only, then sit down	30 sec

Talking points

- **Three themes:** we're measuring the wrong thing; spend less by knowing more; prove return by tying work to outcome.
- **On control:** "We're not removing control. We're upgrading it from a blunt cap to a targeted one."

- **On transition:** "Keep a loose guardrail now. As motion visibility comes online, hard caps become unnecessary."
- **On the data:** "The gateway is almost certainly logging what we need already. Nobody has built the per-motion view yet." Never imply the data is being hidden.
- **On honesty:** label influenced pipeline as an estimate, every time. It buys credibility for the numbers that are hard.
- **On low usage:** "Little AI on a motion is a question, not a virtue — it's either manual work or work we can't see."

Framing rules

- Present it as *how we get what your plan is trying to achieve*, never *your plan is wrong*. The audience should feel rescued, not corrected.
- Pre-wire before the meeting: walk the sponsor through the example privately first, offered as support for her cost-and-ROI story ("you're welcome to use it"). Bring others in afterward, giving owners of adjacent systems a visible win on their own turf.
- Show one undeniable example, mention the larger pattern once, then stop. Don't pitch the full roadmap.

Anticipated leadership questions

Every hard question has a short answer that points back to the one-motion proof.

Question	Answer
So we just let spend run wild?	No. We replace a blind control with a smart one. Visibility lets us cut spend that isn't producing instead of throttling everyone equally.
Where do we start?	One motion, not a platform: the trade-show list. Confirm the data exists (days), build the motion (about two weeks), bring back real numbers.
Where does the measurement take place?	In a derived table in Databricks that joins gateway usage to business objects through the Jira key. Salesforce and Jira stay the sources of truth, joined at query time.

Question	Answer
Where are we now vs. where we need to be?	Raw materials, no assembly. We see spend per model only. Proof is two weeks away; org-wide adoption is quarters, and adoption is the hard part.
What do you need from us?	Read access to the gateway usage logs, and a nod to build one example before committing to role-based dashboards.
Why not just build the role dashboards first?	Role views fall out of the motion model for free — every ticket has an owner. Built the other way, motion views are impossible.
What if the model mangles the IDs?	It never sees them. The ticket key travels as request metadata, not prompt text, and every key is validated against real records.
Isn't this a lot of engineering?	The proof is scripts on one machine. The plugin, hooks and skills library come only after the proof earns it.
How do we get people to adopt it?	Make the governed path the easy path: one command beats building a spreadsheet. Enforcement rides in hooks so nobody has to remember the rules.
What about work that doesn't go through the gateway?	It's invisible to this telemetry, and we'll say so. Mapping what routes around the gateway is part of discovery.
Doesn't low AI usage mean someone is being efficient?	Not necessarily. It may mean manual work — usually the most expensive way — or a tool we can't see. It marks where to look next.
How confident are the revenue numbers?	Goal attainment is defensible. Influenced pipeline is labeled an estimate, and the credit rule (influenced vs. sourced) gets agreed before we report it.

Open questions and first actions

Everything downstream depends on one row: pull a real request from the gateway's usage or inference table before building anything.

First actions

- [ ] Find the gateway's usage table (`system.ai_gateway.usage` or `system.serving.endpoint_usage`) or its inference table in Unity Catalog, and pull one real row
- [ ] From that row, confirm: is logging on, is there a tag field (`request_tags` or `usage_context`), and are payloads intact or null from the 1 MiB cap
- [ ] Test one tagged call: set `ANTHROPIC_CUSTOM_HEADERS` in a project's settings and check the tag lands in the table
- [ ] Stage the ugliest real trade-show list as a table in Databricks; confirm the account and contact tables are queryable
- [ ] Create the Jira ticket and Salesforce campaign (owner, budget, dates, goal) for the proof
- [ ] Build the six-step pipeline for that one list, writing the motion run log as it goes
- [ ] Write the ledger query for that campaign and fill the before/after table
- [ ] Walk the sponsor through the example privately before the meeting

Questions for the gateway owner

Phrase as discovery, never accusation: *"I think the data's already being captured — can you help me find it?"*

1. Which gateway variant do we run, and is usage tracking or inference-table logging enabled on our endpoint?
2. Can we get read access, or a filtered view, on the usage table? It is admin-only by default.
3. Can we expose per-user and per-tool consumption, not just per-model?
4. What else routes around the gateway?

Still open

Question	Why it matters
Who owns the gateway endpoint?	Determines who grants access and whether tagging is allowed
Header tags or body `usage_context`?	Decides how Claude Code and pipeline scripts attach the ticket key
Influenced vs. sourced credit on opportunities	Must be agreed before any revenue number is shown
What routes around the gateway besides Atlassian MCP	Defines the true boundary of what we can measure

Sources

- Databricks: Unity AI Gateway usage tracking
- Databricks: Configure AI Gateway on model serving endpoints
- Claude Code: Hooks reference
- Claude Code: Environment variables